



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 3	Tema: Introducción a JavaScript
	Semana 9	Tutor: Ing. Victoria Castro

Introducción:

¿Qué es JavaScript y cómo funciona con HTML y CSS?

JavaScript es un poderoso lenguaje de programación que aporta interactividad y comportamiento dinámico a los sitios web.

Mientras que HTML y CSS son lenguajes de marcado usados para estructurar el contenido y dar estilo a los elementos de una página, JavaScript va más allá al permitir funcionalidades más complejas, como manejar la entrada del usuario, animar elementos e incluso construir aplicaciones web completas.

Por ejemplo, cuando haces clic en un botón, envías un formulario o pasas el cursor sobre un menú, JavaScript determina cómo se comporta la página.

¿Por qué estudiar JavaScript en Ingeniería de Software? JavaScript ha dejado de ser un simple lenguaje para hacer "efectos visuales" y se ha convertido en una pieza fundamental de la ingeniería por tres razones:

1. **Omnipresencia (Full Stack):** Es el único lenguaje que corre nativamente en el navegador (Frontend) y, gracias a Node.js, también en servidores (Backend).
2. **Ecosistema Robusto:** La gestión de dependencias (npm) y frameworks como React, Angular o Vue permiten construir sistemas escalables y mantenibles.
3. **Asincronía:** Su modelo de eventos permite manejar múltiples tareas (como peticiones a bases de datos) sin bloquear la ejecución del programa, algo vital para el rendimiento de aplicaciones modernas.

Sección 1: Variables y Ámbito

¿Qué son las variables y cuáles son las pautas para nombrar variables en JavaScript?

En JavaScript, las **variables** actúan como contenedores para almacenar datos que puedes acceder y modificar a lo largo de tu programa.

Puedes pensar en las variables como cajas que guardan valores. Con ellas puedes llevar el control de números o textos y referirte a esos valores cuando los necesites en tu código.

Una forma de declarar una variable en JavaScript es usando la palabra clave **let**. Aprenderás más sobre *let* y otras formas de declarar variables en lecciones futuras.

Ejemplo de declaración de una variable llamada *age*:

```
Js ^  
  
let age;
```

En este momento, la variable *age* no tiene un valor asignado. Si intentas usarla, devolverá **undefined**, lo que significa que no tiene valor.

Para asignar un valor a una variable se usa el **operador de asignación (=)**:

```
Js ^  
  
let age = 25;
```

Ahora, cuando uses la variable *age*, devolverá el valor 25.

Reasignación de valores

Una ventaja de usar **let** es que puedes reasignar valores a una variable ya declarada:

```
Js ^  
  
let age = 25;  
age = 30;
```

Ahora la variable *age* guarda el valor 30. No fue necesario volver a escribir *let* porque la variable ya estaba declarada.

La **reasignación** es útil porque permite actualizar valores mientras el programa se ejecuta, por ejemplo, los puntos en un juego.

Pautas para nombrar variables

Nombrar variables parece sencillo, pero existen reglas y buenas prácticas:

- Los nombres deben describir lo que representan.
 - Ejemplo malo:

```
let x = 10;
```

```
let y = "John";
```

- Ejemplo bueno:

```
let age = 10;
```

- Las variables deben comenzar con una letra, un guion bajo `_` o un signo de dólar `$`.
 - Válido:

```
let age;
```

```
let _score;
```

`let $total;`

- o Inválido:

`let 1stPlace; // empieza con número`

- Los nombres son **sensibles a mayúsculas y minúsculas**: `age` y `Age` son variables diferentes.
- Se recomienda usar la convención **camelCase**, donde la primera palabra va en minúscula y las siguientes comienzan con mayúscula:

`let thisIsCamelCase;`

`let anotherExampleVariable;`

`let freeCodeCampStudents;`

- No puedes usar palabras reservadas de JavaScript como `let`, `const`, `function` o `return`.
- Evita caracteres especiales como `!` o `@`. Lo mejor es usar letras, números, guiones bajos o signos de dólar.

Al seguir estas pautas, tu código será más limpio, legible y fácil de mantener conforme crezca en complejidad.

¿Cómo funcionan de manera diferente *let* y *const* en la declaración, asignación y reasignación de variables?

Cuando trabajas con JavaScript, a menudo declaras variables para almacenar datos que planeas usar en tu programa.

En JavaScript moderno, **let** y **const** son las formas preferidas de declarar variables, pero difieren en cómo manejan la asignación y la reasignación de valores.

En esta lección veremos cómo se diferencian en la declaración, asignación y reasignación de variables.

Uso de **let**

La palabra clave **let** permite declarar variables que pueden actualizarse o reasignarse más adelante. Puedes pensar en *let* como un contenedor flexible: una vez que guardas un valor en él, puedes cambiarlo según lo necesites en tu programa.

Ejemplo:

`let score = 10;`

Aquí la variable *score* se declara y se le asigna el valor 10. Si quieres actualizarlo después, puedes hacerlo:

`score = 20;`

Ahora *score* guarda el valor 20. Esto hace que *let* sea útil cuando sabes que el valor de una variable cambiará durante la ejecución del programa.

Uso de const

La palabra clave **const** se usa para declarar variables que son constantes. Una vez que asignas un valor a una variable declarada con *const*, no puedes reasignarlo.

Ejemplo:

```
const maxScore = 100;
```

Si intentas cambiarlo:

```
maxScore = 200; // Error
```

Esto generará un error en la consola, porque las variables declaradas con *const* son inmutables después de su asignación.

Diferencias clave entre let y const

- Con **let** puedes declarar una variable sin asignarle un valor de inmediato y hacerlo más tarde:

```
let age;
```

```
age = 25;
```

- Con **const**, la variable debe recibir un valor en el momento de la declaración. Si no lo haces, obtendrás un error:

```
const age; // Error: falta inicialización
```

¿Cuándo usar cada uno?

- Usa **let** cuando necesites variables que cambien con el tiempo, como puntajes en un juego o valores que se actualizan.
- Usa **const** cuando quieras declarar valores que no deben cambiar, como configuraciones o constantes de tu programa.

Nota sobre var

También existe la palabra clave **var**, que funciona de manera similar a *let*, pero tiene un alcance más amplio y puede causar problemas en tu código. Por eso, hoy en día se recomienda usar **let** y **const** en lugar de *var*.

Sección 2: Tipos de Datos

¿Qué es un tipo de dato y cuáles son los diferentes tipos de datos en JavaScript?

En JavaScript, un **tipo de dato** es la clase de valor que almacenas, como un número o un texto.

Una **variable** es un contenedor con nombre que guarda un valor de un tipo de dato específico, lo que te permite referenciarlo y manipularlo en tu código.

En matemáticas ya trabajabas con variables así:

```
Js ^  
  
x = 2  
y = 4  
  
x + y
```

Podías crear variables como *x* y *y* y luego usarlas en operaciones matemáticas como la suma. En programación es un concepto similar: puedes crear tus propios nombres de variables y asignarles valores. Estos valores serán de diferentes tipos de datos.

Los **tipos de datos** ayudan al programa a entender con qué clase de información está trabajando, ya sea un número, un texto u otra cosa.

JavaScript tiene varios tipos de datos básicos que usarás en tus programas.

1. Number (Número)

Representa tanto enteros como números decimales. Ejemplos de enteros: 7, 19, 90. Ejemplos de decimales: 3.14, 5.2, -14.5.

Nota: **console.log()** es una función que muestra información en la consola del navegador, usada para depurar código. Los símbolos `//` se usan para escribir **comentarios**, que son notas ignoradas al ejecutar el programa.

2. String (Cadena de texto)

Una **String** es una secuencia de caracteres encerrada entre comillas. Ejemplo con comillas dobles: "I love to code!" Ejemplo con comillas simples: 'I love to code!'

Se usan para representar nombres, etiquetas, mensajes de alerta, etc.

3. Boolean (Booleano)

Representa uno de dos valores: true o false. Ejemplo: un programa puede verificar si un usuario está conectado (true) o no (false) y mostrar la página correspondiente.

4. undefined y null

- **undefined**: una variable fue declarada pero aún no tiene valor.
- **null**: la variable fue intencionalmente asignada a "nada".

5. Symbol

Un tipo especial de valor único e inmutable, usado para crear identificadores únicos:

```
Js ^  
  
Symbol('mySymbol');
```

6. BigInt

Se usa para números muy grandes que exceden el límite del tipo Number:

```
Js ^  
  
1234567890123456789012345678901234567890n;
```

Comprender estos tipos de datos te ayuda a manejar diferentes clases de información en tus programas, ya que cada tipo tiene sus propias características y comportamientos.

7. Object (Objeto)

Un objeto es una colección de pares clave-valor:

```
Js ^  
  
{  
  name: "Alice",  
  age: 30  
}
```

Sirven para agrupar información relacionada.

8. Array (Arreglo o Matriz)

Un Array es una lista ordenada de valores. Se define usando corchetes [] y cada elemento tiene una posición numérica llamada **índice** (que siempre comienza en 0).

```
let frutas = ["manzana", "banana", "cereza"];  
  
console.log(frutas[0]); // Muestra "manzana"
```

Los arrays son fundamentales en ingeniería de software para manejar listas de usuarios, productos o cualquier colección de información.

Sección 3: Operadores

¿Cómo procesamos la información?

Los operadores nos permiten manipular valores, realizar cálculos matemáticos y tomar decisiones lógicas.

1. Operadores Aritméticos

Se usan para realizar cálculos matemáticos básicos.

- + (Suma), - (Resta), * (Multiplicación), / (División).
- % (Módulo): Devuelve el residuo de una división.

```
let suma = 10 + 5; // 15
```

```
let residuo = 10 % 3; // 1 (porque 3 cabe 3 veces en 10 y sobra 1)
```

2. Operadores de Comparación

Se usan para comparar dos valores y siempre devuelven un booleano (true o false).

- == (Igualdad débil): Compara el valor, pero no el tipo.
- === (Igualdad estricta): **Recomendado**. Compara tanto el valor como el tipo de dato.
- != (Diferente), > (Mayor que), < (Menor que).

Nota de ingeniería: En JavaScript, siempre prefiere ===. Por ejemplo: 5 == "5" es true, pero 5 === "5" es false. Usar la igualdad estricta evita errores sutiles de lógica.

3. Operadores Lógicos

Permiten combinar múltiples comparaciones.

- **&&** (AND): Es verdadero solo si **ambas** condiciones son ciertas.
- **||** (OR): Es verdadero si al menos **una** condición es cierta.
- **!** (NOT): Invierte el valor (lo que es true pasa a ser false).

Sección 4: Estructuras de Control

Las estructuras de control permiten que tu programa deje de ser una lista lineal de instrucciones y empiece a "tomar decisiones" o "repetir tareas".

1. Condicionales (if, else if, else)

Permiten ejecutar un bloque de código solo si se cumple una condición específica.

```
let hora = 14;  
if (hora < 12) {  
  console.log("Buenos días");  
} else if (hora < 18) {  
  console.log("Buenas tardes");  
} else {  
  console.log("Buenas noches");  
}
```

2. Bucles o Loops (for, while)

Se usan para repetir una acción varias veces sin tener que escribir el código una y otra vez.

- **Bucle for:** Ideal cuando sabes exactamente cuántas veces quieres repetir algo (por ejemplo, recorrer una lista).

```
for (let i = 0; i < 5; i++) {  
  console.log("Este mensaje se repetirá 5 veces");  
}
```

- **Bucle while:** Se repite mientras una condición sea verdadera. Es útil cuando no sabes cuántas vueltas dará el ciclo.

```
let pasos = 0;
```

```
while (pasos < 3) {  
  console.log("Dando un paso...");  
  pasos++;  
}
```

¿Por qué es vital esto en Ingeniería de Software?

Como ingeniero, no solo escribes código para que "funcione", sino para que sea **eficiente** y **escalable**.

- Los **Arrays y Objetos** te permiten modelar datos del mundo real (como una base de datos de clientes).
- Los **Operadores Lógicos** son la base de la arquitectura de reglas de negocio.
- Las **Estructuras de Control** optimizan procesos, permitiendo que el software maneje miles de datos de forma automatizada en milisegundos.

Bibliografía:

Para profundizar en la sintaxis y ver ejemplos técnicos avanzados, consulta la documentación de **MDN Web Docs (Mozilla)** en: <https://developer.mozilla.org/es/docs/Web/JavaScript>